

SYSTEM DESIGN

Chapter 50: System Design Fundamentals

Personal Notes

Chapters 1-49 built out the entire DSA + algorithms toolkit — everything you need for the coding rounds. This chapter starts a NEW domain: system design. At senior levels, Google (and every FAANG) evaluates you in two tracks — coding for the algorithmic thinking, system design for the ability to build systems that serve millions of users. The two are graded independently; you can't skate through system design on strong coding alone.

System design isn't about knowing every technology by name. It's about (1) asking the right clarifying questions to nail down requirements, (2) making trade-offs justified by concrete constraints, and (3) building up a coherent architecture step by step. This chapter covers the FOUNDATIONS — the interview framework, capacity estimation, scaling patterns, caching, databases, load balancing, message queues, and the trade-off vocabulary you'll speak throughout every interview. Later chapters can drill into specific designs (URL shortener, chat, feed, video streaming, etc.).

1. Why System Design Matters

At Google (and most FAANG), interview loops for senior/staff engineers include 1-2 SYSTEM DESIGN rounds — usually 45 minutes each. These are open-ended: "Design Twitter," "Design a URL shortener," "Design a distributed cache." There's no single right answer. What interviewers look for:

- Do you clarify requirements before jumping to solutions?
- Do you estimate scale correctly (QPS, storage, bandwidth)?
- Do you make trade-offs consciously and justify them?
- Can you go deep on one component when asked?
- Do you know the standard building blocks (LB, cache, queue, DB) and their properties?
- Do you handle bottlenecks, failures, and hot spots?

The bar shifts by level: Junior engineers get basic "design a chat app" scenarios. Senior engineers get more complex multi-service designs. Staff and above are expected to lead the discussion, drive the architecture, and articulate why one trade-off wins over another at Google's scale. Understanding this progression tells you how deep to go for your target level.

2. Functional vs Non-Functional Requirements

Every system design starts with these two categories. Nailing them upfront prevents most missteps later.

Type	Description
Functional	What the system does. Features, use cases, user-visible behavior. "Users can post tweets, follow others, view timeline."
Non-functional	How the system does it. Constraints and quality. "Handle 100M DAU, p99 < 200ms, 99.9% uptime, eventual consistency for feeds."

The 5 Non-Functional Dimensions to Always Ask About

- Scale — how many users, requests/sec, data volume?
- Latency — how fast does each operation need to be (p50, p99)?
- Availability — what uptime is acceptable? (99% vs 99.9% vs 99.99%)
- Consistency — strong vs eventual? Can readers see stale data?
- Durability — is data loss acceptable? Under what failures?

Don't skip to the whiteboard: The #1 mistake in system design interviews is jumping to "I'd use Kafka and Redis" before you know what you're building. Spend 5-10 minutes on requirements — the interviewer WILL prompt back with additional constraints that reshape your design. Getting requirements right upfront is half the battle.

3. The Interview Framework (7 Steps)

A repeatable structure for any 45-minute system design interview. Use this every time — it prevents you from getting lost and shows interviewers you have a mental model.

1. REQUIREMENTS — clarify functional + non-functional. 5-8 minutes.
2. CAPACITY ESTIMATION — back-of-envelope math for QPS, storage, bandwidth. 3-5 minutes.
3. API DESIGN — top-level endpoints or method signatures. 3-5 minutes.
4. DATA MODEL — schemas, relationships, key access patterns. 3-5 minutes.
5. HIGH-LEVEL DESIGN — draw the boxes: clients, LB, services, DBs, caches. 10-15 minutes.
6. DEEP DIVES — pick 1-2 components; discuss internals, trade-offs. 10-15 minutes.
7. BOTTLENECKS & SCALING — what breaks first at 10× the load? How do you fix it? 5-10 minutes.

Manage time explicitly: Don't spend 25 minutes on requirements. Set implicit checkpoints — around minute 10 you should have requirements done, around minute 20 the high-level diagram should be up. Interviewers will steer you, but showing self-awareness of pacing signals seniority.

4. Back-of-the-Envelope Capacity Estimation

Every good design has NUMBERS. Not exact ones — order of magnitude is enough. But you must be able to say "we're looking at ~50K QPS write and ~500K QPS read, ~5 TB per year of storage." These numbers drive every subsequent decision.

Numbers Every Engineer Should Know (Approximate)

Operation	Latency
L1 cache reference	~0.5 ns
Branch mispredict	~5 ns
L2 cache reference	~7 ns
Main memory reference	~100 ns
Compress 1 KB with Snappy	~2 µs
Send 1 KB over 1 Gbps network	~10 µs
Read 4 KB from SSD	~150 µs
Read 1 MB sequentially from memory	~250 µs
Round-trip within same datacenter	~500 µs
Read 1 MB sequentially from SSD	~1 ms
Disk seek	~10 ms
Round-trip across US	~150 ms

Estimation Template — QPS + Storage

Example: Design Twitter

Users: 300 M total, 200 M DAU (Daily Active Users)
Tweets/day/user: 2 average
Read:write ratio: 100:1 (people read timelines far more than post)

Write QPS: $200M \times 2 / 86400 \text{ s} \approx 4,600 \text{ tweets/sec}$ (~5K)
Read QPS: $5K \times 100 = 500,000 \text{ timeline reads/sec}$

Storage per tweet: ~300 B (text + metadata)
Daily storage: $400 \text{ M tweets} \times 300 \text{ B} = 120 \text{ GB / day}$
Annual storage: $120 \text{ GB} \times 365 \approx 44 \text{ TB / year}$
10-year plan: ~500 TB (raw), factor 3x for replication → ~1.5 PB

The rule of thumb: Convert everything to per-second numbers. 1 day = ~100K seconds (86,400 to be exact). If a system serves 10M requests/day, that's ~100 QPS average — but PEAK might be 5-10x higher. Always distinguish average and peak.

5. Vertical vs Horizontal Scaling

Aspect	Vertical (Scale UP)	Horizontal (Scale OUT)
Approach	Bigger machine	More machines
Cost curve	Cheap → expensive fast	Linear in machine count
Ceiling	Physical hardware limits	Effectively unlimited
Complexity	Low — same architecture	High — needs distribution logic
Failure impact	Single machine → whole outage	One node fails, others take over
Best for	Databases, small teams	Web servers, stateless workloads

The senior perspective: Most real systems start vertical (simpler) and go horizontal (necessary at scale). The BEST answer in interviews is often "start vertical, migrate to horizontal when we hit X threshold." Showing this staged thinking beats "use Kubernetes from day one."

6. Load Balancing

A LOAD BALANCER (LB) sits in front of a pool of servers and distributes incoming requests across them. It's usually the first layer after DNS/CDN. LBs give you horizontal scale (many servers behind one endpoint) and fault tolerance (route traffic away from failed nodes).

Load Balancing Algorithms

Algorithm	How it works
Round Robin	Rotate through servers sequentially. Simple, fair for uniform workloads.
Least Connections	Send request to the server with fewest active connections. Good for long-lived requests.
Weighted	Assign weights to servers based on capacity — bigger machines get more traffic.
IP Hash	Hash client IP to pick a server. Same IP → same server (useful for session stickiness).
Consistent Hash	Advanced — see Section 12. Minimizes remapping when servers are

Algorithm	How it works
	added/removed.

Layer 4 vs Layer 7

- Layer 4 (TCP-level) — routes based on IP/port only. Fast, protocol-agnostic. Example: AWS NLB.
- Layer 7 (HTTP-level) — inspects headers, paths, cookies. Enables path-based routing (e.g., /api/* → API servers). Slower but more flexible. Example: AWS ALB, nginx.

7. Caching

CACHES store frequently-accessed data closer to the request path, avoiding expensive DB queries or computations. A well-designed cache turns a 100 ms DB query into a 1 ms memory lookup. In practice, caching is the single biggest lever for reducing latency and DB load.

Where to Cache

Cache Layer	Notes
Browser cache	Static assets, HTTP cache headers. Nearest to user.
CDN (edge)	Cache at ISP edge locations. Serves global users quickly.
Reverse proxy cache	nginx/Varnish cache at your edge. Full HTTP responses.
Application cache	In-process cache (Guava, Caffeine in Java). Fastest but per-instance.
Distributed cache	Redis, Memcached. Shared across all app instances.
Database cache	DB's own buffer pool. Automatic; less flexible.

Cache Strategies

- Cache-aside (lazy loading) — app checks cache first; on miss, reads DB and populates cache. Most common pattern.
- Read-through — cache library reads DB automatically on miss. App only talks to cache.
- Write-through — writes go to cache AND DB simultaneously. Consistent but slow writes.
- Write-behind — writes go to cache; DB updated asynchronously. Fast writes, but data-loss risk on crash.

Eviction Policies

- LRU (Least Recently Used) — evict what hasn't been accessed longest. Default in most caches.
- LFU (Least Frequently Used) — evict what's accessed rarely. Better for hot-set workloads.
- TTL (Time To Live) — evict after a fixed time. Guarantees freshness.

Cache invalidation is famously hard: "There are only two hard problems in computer science: cache invalidation and naming things." Whenever data changes, the cache and DB can drift. Common bugs: stale reads, dogpile (cache miss thundering herd on hot key), inconsistent regions. Discuss invalidation strategy in interviews — never say "we cache everything" without addressing this.

8. SQL vs NoSQL Databases

The database decision drives most of your architecture. "SQL or NoSQL?" is one of the first strategic choices in every design. There's no universal right answer — but there are strong signals.

Aspect	SQL (Relational)	NoSQL
Schema	Fixed, enforced	Flexible / schema-less
Query language	SQL — powerful joins, transactions	Custom APIs; typically no joins
Consistency	Strong (ACID)	Eventual (BASE) — usually
Scaling	Vertical primarily; sharding is hard	Horizontal by design
Best for	Relational data, financial, transactional	High-scale, denormalized, flexible schema
Examples	PostgreSQL, MySQL, Oracle, SQL Server	MongoDB, Cassandra, DynamoDB, Redis

NoSQL Sub-Types

- Key-Value: Redis, DynamoDB — simple lookups by key. Very fast.
- Document: MongoDB, CouchDB — JSON-like documents with nested structure.
- Wide-Column: Cassandra, HBase — big-data storage, columnar reads. Great for time-series.
- Graph: Neo4j, ArangoDB — relationships as first-class citizens. Best for social networks.

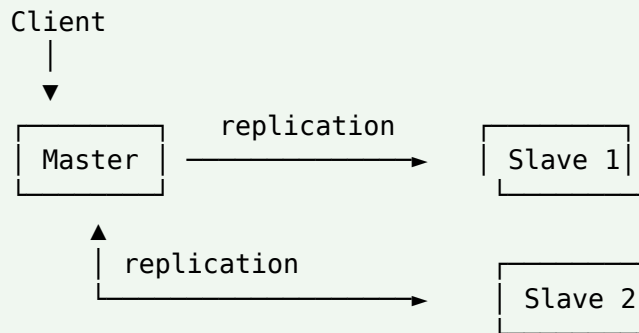
9. Database Replication

Replication copies data across multiple database instances. It's the foundation of both HIGH AVAILABILITY (one node dies, others take over) and READ SCALABILITY (spread reads across replicas).

Common Patterns

- Master-Slave (Single-Leader) — writes go to master, replicated to slaves. Slaves handle reads. Simple, most common.
- Master-Master (Multi-Leader) — multiple write nodes, replicated bidirectionally. Complex conflict resolution.
- Leaderless — any replica accepts writes, uses quorum reads. Dynamo/Cassandra pattern.

Master-Slave (Single-Leader):



Writes: client → master

Reads: client → any slave (higher throughput)

Failover: promote a slave to master when the master dies

Replication lag is the classic gotcha: Slaves are USUALLY behind the master by milliseconds — sometimes seconds under load. This means a user might see their write reflected on the master but not on a slave they read from. If your app requires "read your own writes," you either read from the master, use synchronous replication (slower), or route reads to a slave you know is caught up.

10. Database Sharding

SHARDING partitions a database's data across multiple machines, each holding a subset. Unlike replication (which duplicates), sharding SPLITS. Necessary once your data outgrows a single machine or writes exceed one node's capacity.

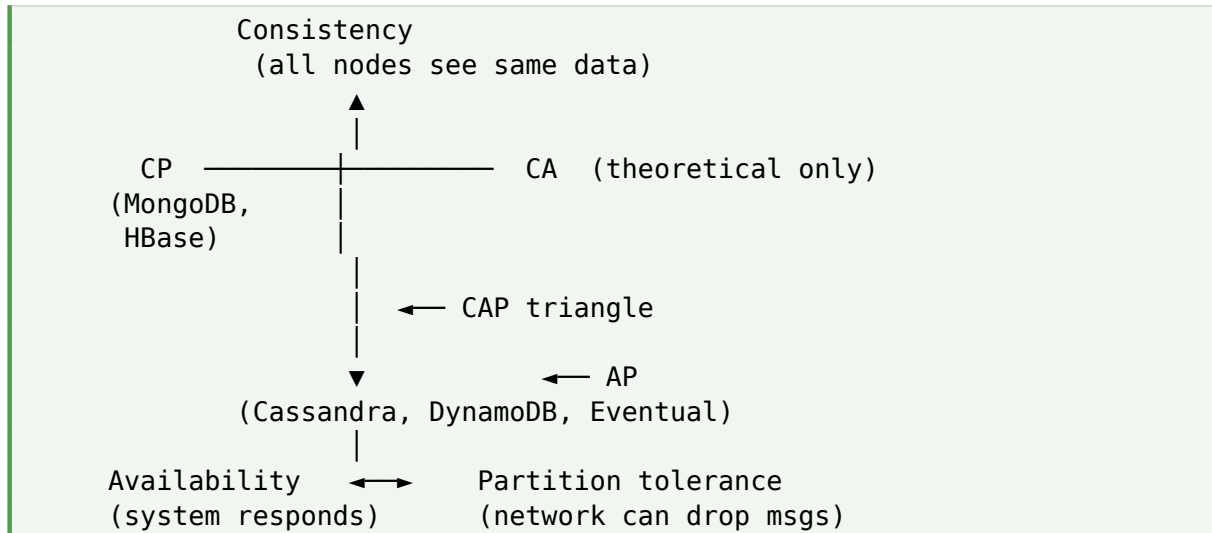
Sharding Strategies

- Range-based — partition by key range (e.g., users A-M in shard 1, N-Z in shard 2). Simple; risk of hot ranges.
- Hash-based — hash the key, mod by shard count. Uniform distribution. Adding/removing shards remaps everything (see consistent hashing).
- Geo-based — partition by user region. Great for locality, but cross-region queries are painful.
- Directory-based — a separate lookup service maps keys to shards. Flexible but adds latency + a new dependency.

The shard key is everything: The BIGGEST decision in sharding is the shard key. Wrong shard key → hot shards, expensive cross-shard queries, painful re-sharding. Choose a key that (a) spreads writes evenly, (b) aligns with your most common read pattern. Twitter shards by `user_id`; Uber by `driver_id` or geohash.

11. CAP Theorem

CAP theorem: in a distributed system, you can guarantee at most TWO of these three properties: Consistency, Availability, Partition tolerance. Since network partitions are inevitable in real systems, the real choice is between CONSISTENCY and AVAILABILITY when a partition happens.



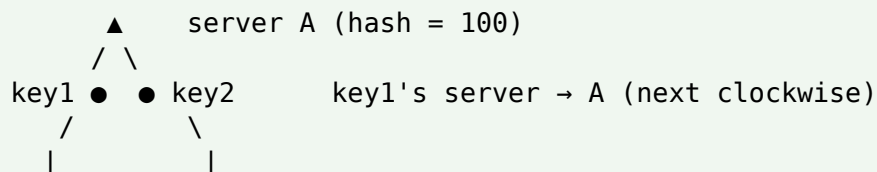
- CP (Consistency + Partition tolerance) — reject requests during partition to keep data consistent. Example: MongoDB, HBase, ZooKeeper.
- AP (Availability + Partition tolerance) — respond during partition even if data is stale. Example: Cassandra, DynamoDB, Riak.
- CA — only possible when NO partition. In practice, unreachable — networks fail.

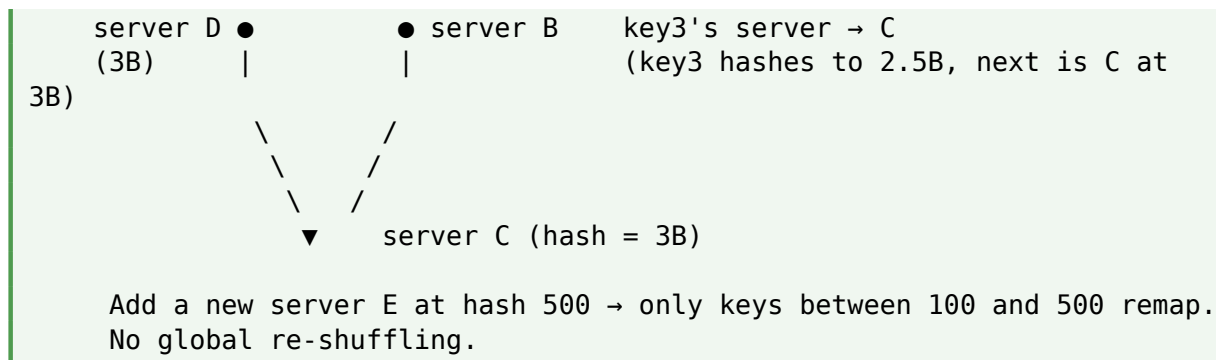
PACELC extends CAP: CAP is oversimplified — it only describes what happens DURING partitions. In practice, even without partitions, you trade LATENCY for CONSISTENCY. PACELC ("if Partition, A or C; Else, Latency or C") captures this better. For interviews, mention PACELC to show depth: "Cassandra is AP-EL — favors availability under partitions and low latency otherwise."

12. Consistent Hashing

Naive hash sharding (hash % N) has a fatal flaw: when N changes, EVERY key remaps to a new shard. Adding one server → move ~all data. CONSISTENT HASHING solves this: keys and servers hash to points on a ring; a key belongs to the next server clockwise. Adding a server only remaps keys in a small arc — not everything.

Hash ring (values 0 to $2^{32} - 1$, wrapping):





Virtual Nodes

One server = one point → uneven distribution (some servers get more keys than others). Fix: each server maps to MANY "virtual nodes" (e.g., 200 hash positions). Uniform distribution emerges from the law of large numbers.

- Used by: Amazon Dynamo, Apache Cassandra, memcached client libraries, most modern distributed hash tables

13. Message Queues

A MESSAGE QUEUE decouples producers and consumers. Producers publish messages; consumers process them asynchronously. Queues absorb spikes, enable retry logic, and let you scale producers and consumers independently — critical for reliable systems.

Kafka vs RabbitMQ (The Two Big Ones)

Aspect	Kafka	RabbitMQ
Model	Distributed commit log	Traditional broker
Ordering	Per-partition, strict	Per-queue, no partition ordering
Persistence	Days of history retained	Delete after ack
Throughput	Very high (millions/sec)	High (100Ks/sec)
Consumers	Pull-based; consumers track offset	Push-based; broker manages
Best for	Event streaming, analytics, replayable	Task queues, RPC, complex routing

When to Add a Queue

- Producer speed >> consumer speed (absorb the burst)
- Consumer might fail and need to retry (durability)
- Multiple consumers need the same event (pub/sub)
- You want to decouple deployment/scaling of producer and consumer

14. CDN & Edge Networks

A CONTENT DELIVERY NETWORK (CDN) caches static content at geographically distributed edge servers, close to end users. Instead of every user hitting your origin server in Virginia, users in Tokyo hit an edge in Tokyo — massively reducing latency and origin load.

- Static assets: images, JS, CSS, videos → almost always CDN-served
- Dynamic content: some CDNs (Cloudflare Workers, Lambda@Edge) run code at the edge
- DDoS protection: CDNs absorb attacks before they reach origin
- Examples: Cloudflare, Fastly, Akamai, AWS CloudFront

Cache-Control headers matter: CDN behavior is driven by HTTP headers (Cache-Control, ETag, Expires). If your origin sets Cache-Control: max-age=86400, the CDN caches for a day. This is one of the highest-ROI performance improvements — often changes latency from 300 ms to 20 ms with a config change.

15. Communication Protocols

Protocol	Style	Best for
REST	Stateless HTTP + JSON	Public APIs, CRUD services
gRPC	HTTP/2 + protobuf, bidirectional	Internal microservices, high performance
GraphQL	Client queries specific fields	Aggregating multiple back-ends, mobile
WebSocket	Full-duplex over one connection	Chat, live updates, gaming
Long polling	Client repeatedly asks for updates	Fallback when WebSocket unavailable
Server-Sent Events (SSE)	Server → client stream	Notifications, dashboards

16. Rate Limiting

RATE LIMITING caps how many requests a client (or API key, or IP) can make per unit time. Prevents abuse, protects against DDoS, keeps costs predictable, and prevents a single misbehaving client from overwhelming the system.

Common Algorithms

- Token Bucket — bucket holds N tokens, refills at rate R/sec. Each request consumes a token. Rejected if empty. Simple + allows bursts.
- Leaky Bucket — requests fill a bucket; leaked out at fixed rate. Smooths bursts.

- Fixed Window — count requests in each 1-minute window. Simple; risk of bursts at boundaries.
- Sliding Window Log — timestamp of every request; count how many in past N seconds. Accurate but memory-heavy.
- Sliding Window Counter — combines fixed windows with weighted overlap. Google's approach.

Where to place rate limiting: Ideally at multiple layers — at the LB or API gateway (basic per-IP protection), plus in each service (per-user, per-API-key). Storing counters in Redis is the standard implementation. For interviews, mention the Token Bucket algorithm — it's the most common and easiest to explain.

17. Common Trade-offs Vocabulary

System design is about trade-offs. Here's the language you'll use to justify every decision:

Trade-off	What you're choosing between
Latency vs Throughput	Batch processing = higher throughput but higher latency per item.
Consistency vs Availability	CAP — during partition, you pick one.
Read-optimized vs Write-optimized	Denormalize for reads; normalize for writes.
Cache vs Freshness	Bigger TTL = fewer misses but staler data.
Vertical vs Horizontal scale	Simplicity vs unbounded ceiling.
Push vs Pull	Push = real-time but requires client to be online; pull = simpler, higher latency.
Synchronous vs Async	Sync = simple but couples services; async = decoupled but complex failure modes.
Stateless vs Stateful	Stateless = easy to scale but pushes state elsewhere; stateful = simpler locally, harder to scale.

18. Common Mistakes

Mistake 1: Skipping requirements

Jumping to "I'd use microservices with Kafka and Redis" before you know QPS, data size, or the actual use case. Always spend 5-10 minutes clarifying. Ask about scale, latency, consistency, availability.

Mistake 2: Over-engineering for scale you don't have

Not every problem needs Kafka, Kubernetes, and 10 microservices. For a 1000-user internal tool, a monolith with PostgreSQL is the RIGHT answer. Match complexity to scale.

Mistake 3: Under-engineering the deep dive

When asked "how would you shard the DB?" — the answer isn't "we shard it." Discuss the shard key, hot spots, cross-shard queries, resharding strategy.

Mistake 4: Naming technologies without knowing them

If you say "I'd use ZooKeeper for coordination," expect the follower to ask HOW. Don't cite tools you can't defend. If unsure, describe the CAPABILITY ("a strongly consistent metadata store") rather than the brand.

Mistake 5: Ignoring failure modes

What happens if the DB dies? The cache is empty? The queue backs up? Real-world systems fail — your design should have a story for the top 3 failure modes.

Mistake 6: Not managing time

Spending 30 minutes on requirements and 5 minutes on the actual design. Set implicit checkpoints — around minute 10, requirements done; minute 25, first draft of the architecture; minute 40, discussing bottlenecks.

Mistake 7: Silent design

Working alone at the whiteboard for 15 minutes without talking. Interviews are collaborative — think out loud, invite pushback, explain your reasoning. The interviewer wants to see HOW YOU THINK, not just the final answer.

19. Best Practices

- Start every interview with 5-10 minutes on requirements — resist the urge to draw boxes early
- Do capacity estimation on the whiteboard — even rough numbers demonstrate rigor
- State assumptions explicitly ("Assuming 100M DAU and 5:1 read/write ratio...")
- Draw a clean high-level diagram before drilling into details
- For every technology choice, articulate WHY vs the alternatives
- Discuss failure modes proactively — don't wait for the interviewer to ask
- Mention monitoring and observability (metrics, logs, alerts) briefly — shows production experience
- End with a summary: "To recap, we've built X to handle Y load with Z consistency guarantees"

20. Quick Reference

Concept	Meaning
Functional requirements	What the system does (features, APIs)

Concept	Meaning
Non-functional requirements	How well it does (scale, latency, availability)
QPS	Queries Per Second — key capacity metric
Load balancer	Distributes traffic across servers
Cache-aside	App checks cache, populates on miss (most common)
ACID	Atomicity, Consistency, Isolation, Durability (SQL)
BASE	Basically Available, Soft state, Eventual consistency (NoSQL)
Replication	Copy data across nodes for HA + read scale
Sharding	Split data across nodes for write scale
CAP theorem	Pick 2 of Consistency, Availability, Partition tolerance
Consistent hashing	Hash keys and servers to a ring; minimizes remapping
Message queue	Decouples producers from consumers, absorbs bursts
CDN	Edge cache for static assets, close to users
Token bucket	Common rate-limiting algorithm — allows controlled bursts
Read-your-writes	Consistency model — user sees their own writes immediately

21. Practice Problems

These are classic system design interview problems. Practice by walking through the 7-step framework for each — set a 45-minute timer, whiteboard the whole thing.

Beginner (Warm-ups)

8. Design a URL shortener (like bit.ly).
9. Design a rate limiter.
10. Design a key-value store.
11. Design a distributed cache (like Redis).
12. Design a web crawler.

Intermediate (Most Common)

13. Design Twitter / a news feed.
14. Design Instagram / a photo-sharing service.

15. Design a chat system (like WhatsApp or Slack).
16. Design YouTube / a video streaming service.
17. Design Uber / a ride-sharing service.
18. Design a notification system.
19. Design a search autocomplete.

Advanced (Senior/Staff Level)

20. Design a global distributed file system (like GFS or Dropbox).
21. Design a real-time collaborative editor (like Google Docs).
22. Design a payment processing system.
23. Design a stock trading platform.
24. Design an ad-serving platform.
25. Design a distributed job scheduler.
26. Design a metrics and monitoring system (like Datadog).

22. Final Takeaway

System design is a fundamentally different skill from DSA. Coding rounds test whether you can build ONE thing correctly. System design tests whether you can compose MANY things — services, databases, caches, queues, load balancers — into an architecture that serves millions of users reliably. There's no single right answer; interviewers evaluate your REASONING, TRADE-OFF AWARENESS, and DEPTH OF KNOWLEDGE.

For your first system design interview, master three things: (1) the 7-step framework, (2) the standard building blocks (LB, cache, queue, SQL vs NoSQL, replication, sharding), and (3) capacity estimation. That's enough for a solid junior-to-mid interview. Senior+ interviews require deeper knowledge of consistency models, CAP/PACELC, consistent hashing, and specific system designs (URL shortener, chat, feed). Chapter 51+ will drill into these.

Key Takeaway: System design = 7-step framework: REQUIREMENTS → CAPACITY → API → DATA MODEL → HIGH-LEVEL → DEEP DIVE → BOTTLENECKS. Always start with functional + 5 non-functional dimensions (scale, latency, availability, consistency, durability). Standard building blocks: LOAD BALANCER, CACHE (cache-aside, LRU), DATABASE (SQL for ACID, NoSQL for scale), REPLICATION (HA + reads), SHARDING (write scale), MESSAGE QUEUE (decouple + absorb bursts), CDN (edge caching). CAP: during partitions, pick C or A. Consistent hashing minimizes remapping. Always articulate trade-offs.

23. What's Next?

With foundations covered, the natural next steps are specific system designs and deeper dives:

- Design a URL Shortener — the canonical warm-up problem, deeply worked through

- Design a News Feed / Twitter — feed generation, timeline algorithms, celebrity problem
- Design a Chat System — WebSocket architecture, presence, message delivery guarantees
- Design a Video Streaming Service — CDN, chunked encoding, adaptive bitrate
- Design Uber — geohashing, driver-rider matching, real-time updates
- Deep dive: Distributed Consensus — Raft, Paxos
- Deep dive: Databases Internals — B-trees, LSM trees, WAL
- Deep dive: Kafka Architecture — partitions, consumer groups, ISR